Go: тестирование, CI и HTTP API

Необходимо разработать небольшое приложение на Go, последовательно пройдя несколько этапов: реализация функции, автоматическое тестирование, настройка CI и преобразование программы в HTTP-сервис.

Принцип выполнения. Технические решения должны опираться на официальную документацию Go, стандартной библиотеки и GitHub Actions. Готовые решения в задании намеренно не приводятся.

1. Расчёт количества дней до Нового года

Реализуйте функцию, которая определяет количество календарных дней, оставшихся от заданной даты до ближайшего следующего Нового года.

Функция должна принимать дату и возвращать количество дней до 1 января следующего календарного года. Необходимо учитывать особенности календаря, в том числе високосные годы.

Вычислительная логика должна быть отделена от пользовательского интерфейса, HTTP и других способов взаимодействия с программой.

Официальная документация

- Пакет time
- Стандартная библиотека Go

2. Unit-тестирование

Для реализованной функции необходимо написать автоматические unit-тесты. Функцию следует проверить как минимум в трёх различных точках.

Тестовые случаи выбираются самостоятельно. Необходимо подобрать их таким образом, чтобы они проверяли не только обычный случай, но и характерные для задачи границы.

- начало календарного года;
- конец календарного года;
- високосный год;
- даты до и после 29 февраля.

Тестирование должно выполняться стандартными средствами Go. Желательно изучить подход table-driven tests и самостоятельно определить, подходит ли он для данной задачи.

Официальная документация

- Пакет testing
- How to Write Go Code: Testing
- Команда go test

3. Continuous Integration

Проект необходимо разместить в GitHub и настроить автоматическую проверку кода с использованием GitHub Actions.

При отправке изменений в основную ветку репозитория должен автоматически запускаться CI workflow. Также рекомендуется запускать проверки при создании и изменении Pull Request.

Официальная документация

- GitHub Actions
- Workflow syntax for GitHub Actions
- Building and testing Go

Форматирование

Код должен соответствовать стандартному форматированию Go. CI должен обнаруживать наличие исходных файлов, форматирование которых отличается от стандартного.

Официальная документация

- gofmt
- go fmt

Статический анализ

Код должен проходить стандартную статическую проверку Go. Допускается использование дополнительных анализаторов, однако стандартные средства Go должны оставаться частью проверки.

Официальная документация

- go vet
- Пакет cmd/vet

Зависимости

Необходимо проверить корректность зависимостей Go-модуля. Изучите назначение команд go mod tidy и go mod verify. CI должен позволять определить ситуацию, когда состояние зависимостей проекта не соответствует содержимому исходного кода.

Дополнительно рекомендуется исследовать механизм проверки зависимостей на известные уязвимости.

Официальная документация

- Go Modules Reference
- Managing dependencies
- Команда go mod
- Govulncheck tutorial

Сборка

Необходимо автоматически проверять, что проект собирается.

Официальная документация

- Команда go build

Тестирование в CI

Все разработанные тесты должны автоматически запускаться в CI. Если хотя бы одна обязательная проверка завершается ошибкой, workflow не должен считаться успешно выполненным.

Официальная документация

- Команда go test
- Пакет testing

4. Требования к стилю Go-кода

Необходимо изучить принятые в Go правила оформления и организации исходного кода и применять их в проекте. Задача этого этапа — не сформировать собственный стиль оформления, а научиться следовать соглашениям используемого языка.

Обратите внимание как минимум на:

- именование;
- оформление пакетов;
- работу с ошибками;
- комментарии;
- экспортируемые идентификаторы;
- форматирование;
- организацию импортов.

Официальная документация

- Go Documentation
- Effective Go
- Go Code Review Comments
- Go Doc Comments

5. HTTP API

Следующий этап — превратить программу в сетевой сервис. Сервис должен предоставлять API, позволяющее получить количество дней, оставшихся до Нового года.

Способ организации HTTP API, конкретные пути и формат ответа выбираются самостоятельно.

Обязательное поведение:

- запрос без указанной даты выполняет расчёт относительно текущей даты;
- существует возможность передать дату, относительно которой выполняется расчёт;
- формат передачи даты явно определён;
- сервис возвращает машиночитаемый ответ;
- некорректные входные данные корректно обрабатываются;
- HTTP status code соответствует результату обработки запроса.

Для реализации сервиса рекомендуется сначала изучить возможности стандартной библиотеки Go. Использование стороннего HTTP framework допустимо, но студент должен понимать, какие возможности предоставляет используемая библиотека и зачем она нужна.

Сама функция расчёта дней до Нового года не должна зависеть от HTTP.

Официальная документация

- Пакет net/http
- Пакет encoding/json
- Пакет time

6. Интеграционное тестирование API

К unit-тестам функции необходимо добавить интеграционный тест HTTP API. Тест должен обращаться к работающему HTTP handler через HTTP и проверять фактически полученный ответ.

Необходимо проверить как минимум:

- выполнение HTTP-запроса;
- HTTP status code;
- разбор полученного ответа;
- корректность рассчитанного значения.

Интеграционный тест также должен автоматически запускаться через стандартный механизм go test и входить в CI.

Официальная документация

- Пакет net/http/httptest
- Пакет net/http
- Пакет testing

7. Документация проекта

Репозиторий должен содержать README.md, позволяющий другому разработчику понять, как работать с проектом без изучения его исходного кода.

В README необходимо описать:

- назначение приложения;
- требования для его запуска;
- способ сборки;
- способ запуска;
- способ запуска тестов;
- предоставляемый HTTP API;
- формат входных параметров;
- формат ответа.

Команды и способы запуска должны соответствовать реально работающему проекту.

Официальная документация

- Documenting Go code
- About README files (GitHub Docs)

Обязательный результат

В репозитории должны присутствовать:

- ☐ реализация функции расчёта дней до Нового года;
- ☐ не менее трёх unit-тестов;
- ☐ Go-модуль;
- ☐ HTTP API;
- ☐ поддержка расчёта относительно переданной даты;
- ☐ корректная обработка ошибочного ввода;
- ☐ интеграционный тест HTTP API;
- ☐ GitHub Actions workflow;

- ☐ проверка форматирования;
- ☐ статический анализ;
- ☐ проверка зависимостей;
- ☐ автоматическая сборка;
- ☐ автоматический запуск тестов;
- ☐ README с инструкцией по работе с проектом.

На момент сдачи обязательные проверки CI должны успешно проходить.

Дополнительная часть

После выполнения основной части можно расширить CI и сам проект.

Покрытие тестами

Изучите механизм измерения покрытия Go и добавьте соответствующую проверку или отчёт.

Официальная документация

- Coverage for Go applications

Race Detector

Изучите встроенный в Go детектор гонок и добавьте соответствующую проверку в CI, если она применима к проекту.

Официальная документация

- Data Race Detector

Проверка уязвимостей

Добавьте govulncheck в автоматические проверки.

Официальная документация

- Govulncheck tutorial

Контейнеризация

Подготовьте Dockerfile, позволяющий собрать и запустить сервис в контейнере.

Официальная документация

- Dockerfile overview

Дополнительный HTTP endpoint

Добавьте endpoint для проверки состояния сервиса и обоснуйте выбранный формат ответа.

Официальная документация

- Package net/http

Логирование

Изучите стандартный пакет структурированного логирования и добавьте журналирование запросов.

Официальная документация

- Пакет log/slog

Benchmark

Изучите поддержку benchmark-тестов в стандартном пакете testing и добавьте benchmark для основной вычислительной функции.

Официальная документация

- Benchmarks в пакете testing

Результат: реализация → unit-тесты → HTTP API → интеграционные тесты → форматирование → статический анализ → сборка → CI